

Quantum computing fundamentals: Potential improvements for classical computational problems

Denis Mathan

Ostbayrische Technische Hochschule Regensburg, Germany
`denis.mathan@hs-regensburg.de`

Abstract. Quantum computing promises computational advantages over classical approaches, but the nature and extent of these advantages vary significantly across problem classes. This paper examines two representative cases through the lens of computational complexity theory to contrast these differences: unstructured search spaces, addressed by Grover’s algorithm, and integer factorization in $\text{NP} \cap \text{co-NP}$, addressed by Shor’s algorithm. To define the theoretical boundary, the general NP-complete class is also discussed.

1 Introduction

In 1936, Alan Turing introduced the theoretical foundation of modern computing. An abstract machine capable of executing any algorithm by reading and writing symbols on an infinite tape [10]. Five years later, Konrad Zuse completed the Z3. This was the world’s first freely programmable computer, built from relays in wartime Berlin [7]. What followed was one of the most remarkable trajectories in the history of science. Computing power doubling roughly every two years, a pattern so reliable it became known as Moore’s Law [6, p. 4].

So while hardware engineers made transistors smaller and faster, computer scientists developed increasingly efficient algorithms, that reduced the number of operations required to solve a given problem. Together, these two forces transformed computers from room-sized curiosities into ubiquitous tools capable of feats unimaginable to Turing or Zuse.

Yet this progress was never uniform across all problem types. For a particular class of problems, those in which the number of possible solutions grows exponentially with input size, neither faster hardware nor cleverer algorithms have ever produced a structural breakthrough. The Travelling Salesman Problem and satisfiability problems share a characteristic limitation: the best known classical algorithms remain exponential in the worst case. Even intermediate problems like integer factorization only admit sub-exponential classical algorithms, and decades of research have not changed that fundamental picture [2]. Hardware makes these problems faster to run, but not easier to solve in any meaningful sense.

This limitation has become more pressing precisely as hardware progress itself begins to slow. Transistors have shrunk to the scale of individual atoms, and quantum mechanical effects have become a fundamental physical barrier to further miniaturization [9]. The era in which hardware progress could be taken for granted is drawing to a close.

The same physics that limits classical hardware offers a different kind of opportunity. Rather than fighting quantum effects, quantum computing embraces them. By encoding information in qubits, which can exist in superpositions of states and become entangled with one another, quantum computers can execute certain algorithms in ways that have no classical equivalent [6]. The question is not whether quantum computing is faster in general but whether it can offer a structural advantage for precisely those problem classes where classical computing has been stuck for decades.

This paper therefore asks: Why does quantum computing provide an exponential speedup for specific problems, and can this advantage be transferred to general NP-complete problems? To address this question, this paper follows a qualitative literature review methodology, synthesizing foundational works in quantum algorithms and computational complexity theory.

2 Methodology

This paper follows a qualitative literature review methodology, drawing primarily on foundational theoretical works in quantum algorithms and computational complexity theory. Sources were selected based on their canonical status in the field: Nielsen & Chuang for quantum computing fundamentals, Arora & Barak for complexity theory, and the original algorithm papers by Grover and Shor. Three case studies were chosen to span the full spectrum of quantum advantage. Unstructured search as a baseline quadratic speedup, integer factorization as the strongest known example of exponential quantum advantage, and NP-complete problems as the theoretical boundary where quantum advantage provably breaks down. The analysis is deliberately restricted to provable, asymptotic speedups within a complexity-theoretic framework. Heuristic approaches such as the Quantum Approximate Optimization Algorithm (QAOA) or Quantum Annealing are excluded as they do not yield formal complexity guarantees. Similarly, hardware-specific considerations such as decoherence, gate fidelity, or limitations of Noisy Intermediate-Scale Quantum (NISQ) devices are out of scope, as the paper focuses on theoretical complexity rather than practical implementability.

3 Related Work

The relationship between quantum and classical computational complexity has been approached from several directions in the literature. Bernstein and Vazirani [4] formally introduced the complexity class BQP and established its relation to classical complexity classes, providing the theoretical foundation for evaluating quantum algorithms within a complexity-theoretic framework. Bennett et al. [3]

leveraged this framework to demonstrate that Grover’s algorithm is mathematically optimal for unstructured search, proving that the quadratic advantage cannot be exceeded without exploiting underlying structural properties.

Building organically upon the premise that problem structure defines quantum limits, Aaronson and Ambainis [1] formally investigate the relationship between problem structure and quantum speedup, arguing that superpolynomial quantum advantages are fundamentally limited to problems exhibiting algebraic regularity. This modern perspective aligns with and theoretically grounds the structure-dependent view of quantum advantage developed in this paper.

The primary algorithmic works examined in this paper to illustrate these bounds are Grover [5], who introduced the quadratic speedup for unstructured search, and Shor [8], who demonstrated an exponential quantum speedup for integer factorization via period finding. These works are complemented by standard complexity-theoretic references on the placement of factorization in $\text{NP} \cap \text{co-NP}$ and on the boundaries of BQP.

4 Complexity Theory as a Framework

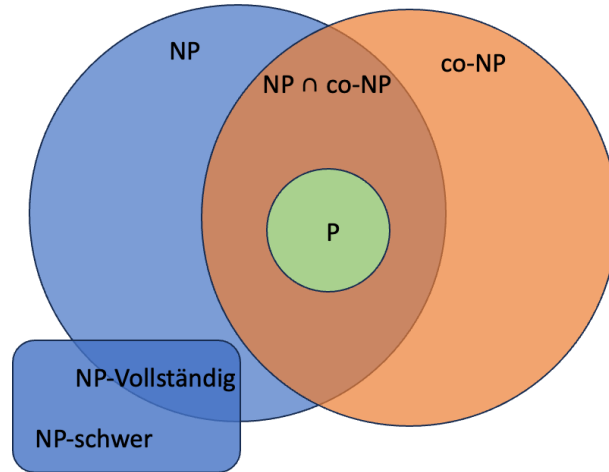


Fig. 1. Visual representation of complexity classes, synthesized from [2,4]. Note that the exact separations depicted here reflect current scientific conjectures rather than strictly proven boundaries.

To assess quantum advantage, one must define difficulty. The relevant resource is the number of elementary steps required to solve a problem as a function of the input size n . This hardware-independent metric is captured by complexity classes [2].

4.1 Classical Complexity: $\mathbf{P}$, $\mathbf{NP}$ and $\mathbf{co-NP}$

The class $\mathbf{P}$ contains decision problems solvable deterministically in polynomial time $\mathcal{O}(n^k)$. These are conventionally considered tractable (e.g., sorting, primality testing). $\mathbf{NP}$ (Nondeterministic Polynomial time) contains problems where a proposed solution can be verified in polynomial time which includes $\mathbf{P}$. Within $\mathbf{NP}$, **$\mathbf{NP-complete}$** problems form the hardest subset. An efficient algorithm for one would efficiently solve all of $\mathbf{NP}$ according to the fact that any problem in $\mathbf{NP}$ can be reduced to it in polynomial time. **$\mathbf{co-NP}$** contains problems where a *negative* answer can be efficiently verified.[2]

The intersection $\mathbf{NP} \cap \mathbf{co-NP}$ represents a structural middle ground: these problems admit efficient verification of both "yes" and "no" instances. So it is given that $\mathbf{P}$ lies in $\mathbf{NP} \cap \mathbf{co-NP}$ [2].

4.2 Probabilistic and Quantum Complexity: $\mathbf{BPP}$ and $\mathbf{BQP}$

Classical complexity also includes **$\mathbf{BPP}$** (Bounded-error Probabilistic Polynomial time), containing problems solvable by randomized algorithms in polynomial time with an error probability bounded by $1/3$ [2]. $\mathbf{BPP}$ is the natural conceptual baseline for quantum computing.

The quantum analogue is **$\mathbf{BQP}$** (Bounded-error Quantum Polynomial time), containing all problems solvable by a quantum computer in polynomial time. It is known that $\mathbf{P} \subseteq \mathbf{BPP} \subseteq \mathbf{BQP}$. It is also widely conjectured, though not formally proven, that $\mathbf{NP} \not\subseteq \mathbf{BQP}$. If this conjecture holds, it means quantum computers do not possess the power to efficiently solve general $\mathbf{NP-complete}$ problems [4,2].

5 Fundamentals of Quantum Computing

Classical computers represent information as bits. Those are binary values that are either 0 or 1 at any given moment. Quantum computing builds on a fundamentally different model of information, one governed by the principles of quantum mechanics. This section introduces the core concepts necessary to follow the algorithmic analyses in Section 6.

5.1 Qubits, Superposition, and Measurement

The basic unit of quantum information is the qubit. Unlike a classical bit, a qubit can exist in a superposition of both 0 and 1 simultaneously. This is described via Dirac (Bra-Ket) notation, a standard mathematical shorthand in quantum mechanics where states are represented as vectors within angle brackets, forming the state vector:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle \tag{1}$$

where α and β are complex amplitudes satisfying $|\alpha|^2 + |\beta|^2 = 1$ [6, p. 13]. The coefficients $|\alpha|^2$ and $|\beta|^2$ represent the probabilities of measuring the qubit in

state $|0\rangle$ or $|1\rangle$ respectively. A system of n qubits can represent 2^n states simultaneously. So for example a register of 8 qubits $|0\rangle^8$ brought into superposition by eight Hadamard gates $H^{\otimes 8}|0\rangle^8$ already spans 256 states at once. This exponential capacity forms the basis of quantum parallelism, allowing quantum operations to theoretically act on all 2^n states in a single developmental step.

Upon measurement, the superposition collapses to a single classical outcome with probability given by the corresponding amplitude squared. This means that the goal of a quantum algorithm is not to maintain superposition indefinitely, but to manipulate amplitudes such that the correct answer has near-certain probability at the moment of measurement. [6]

5.2 Unitary Operators (Quantum Gates)

Quantum computations are performed by applying quantum gates. Operations that transform qubit states. All quantum gates are represented by unitary matrices U , satisfying $U^\dagger U = I$, which guarantees that the total probability across all states is preserved and that every operation is reversible [6, p. 17]. This reversibility is a fundamental requirement of quantum mechanics. No information is destroyed during a quantum computation.

An important example of such an operator is the Hadamard gate, which we already used before creating a superposition from a basis state. The Hadamard gate is represented by the matrix:[6, 31-32]

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \quad (2)$$

5.3 Entanglement

When two qubits interact through a quantum gate, they can become entangled. In this state the measurement outcome of one qubit instantaneously determines the outcome of the other, regardless of physical separation. An entangled two-qubit state cannot be described as two independent single-qubit states. The system must be treated as a whole, usually represented mathematically by the tensor product (denoted by the operator $\otimes$) combining the individual vector spaces. A canonical example is the Bell state. It can be created by applying a

Hadamard gate to the first qubit, then a CNOT = $\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$ gate to entangle

it with the second. Besides using the matrix representation we can also write the state transformation mathematically (using $\otimes$ to construct the combined two-qubit state, where the CNOT gate uses the first qubit as control to decide if the second qubit is going to be flipped) as follows:

$$\text{CNOT}(H|0\rangle \otimes |0\rangle) = \text{CNOT}\left(\frac{1}{\sqrt{2}}(|00\rangle + |10\rangle)\right) = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle) = |\Phi^+\rangle \quad (3)$$

Measuring the first qubit as 0 guarantees the second is also 0, and vice versa.[6, p. 25 - 26]

5.4 Classes of Quantum Algorithms and Speedup Potentials

Quantum algorithms can be broadly categorized into distinct families based on the mathematical structures they exploit and the theoretical speedups they offer over classical counterparts.

- **Amplitude Amplification:** Algorithms in this class, most notably Grover’s algorithm, utilize a diffusion operator to iteratively invert amplitudes about their mean. This concentrates probability mass onto a target state within an unstructured search space. Because they do not rely on problem-specific structure, these algorithms typically yield a broad but strictly limited quadratic speedup (e.g., from $\mathcal{O}(N)$ to $\mathcal{O}(\sqrt{N})$). This limit is mathematically proven for unstructured search [6, chapter 6.6].
- **Quantum Fourier Transform (QFT) and Hidden Subgroups:** This family includes Shor’s algorithm. These algorithms rely on entanglement to evaluate global properties of a function simultaneously. By applying the QFT, they use constructive and destructive interference to extract hidden periodicities. When a problem possesses this specific algebraic regularity, QFT-based algorithms can achieve exponential speedups, shifting problems from classical super-polynomial time into BQP [8].

6 Results: Quantum Improvements by Problem Class

6.1 Unstructured Search: Grover’s Algorithm

Given a set of N elements with no exploitable ordering or structure, find the unique element satisfying a given condition. Classically, this requires examining elements sequentially, yielding an average case complexity of $\mathcal{O}(N)$. No classical algorithm can do better in the general case, since the absence of structure eliminates any possibility of pruning the search space.

Grover’s algorithm solves this problem in $\mathcal{O}(\sqrt{N})$ steps, achieving a quadratic speedup over the best classical approach [5]. The algorithm proceeds in four stages. First, a quantum register of $n = \log_2(N)$ qubits is initialized and brought into a uniform superposition $|s\rangle$ of all N possible states via the Hadamard transform, such that each state carries equal amplitude $1/\sqrt{N}$. Second, a problem-specific oracle operator U_f is applied, which identifies the target state by inverting its phase while leaving all other states unchanged. Crucially, the oracle does not search for the solution. It acts as a verification function, flipping the phase of any state that satisfies the search condition.

Third, a diffusion operator is applied, which inverts all amplitudes about their mean, as illustrated in Figure 2. Crucially, this reflection operates on the quantum amplitudes (α) rather than probabilities ($|\alpha|^2$), enabling constructive

interference. Because the target state carries a negative amplitude after the oracle, it lies furthest from the mean and is therefore amplified disproportionately when mirrored, while the non-target amplitudes are suppressed. This combination of phase inversion and amplitude amplification constitutes a single Grover iteration.

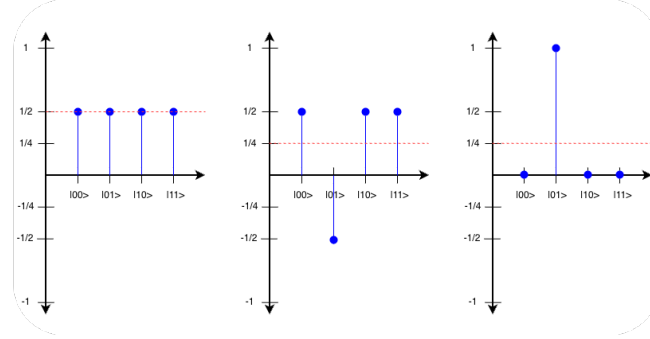


Fig. 2. Visualization of Grover's amplitude amplification for $N = 4$ states. The oracle inverts the amplitude of the target state ($|01\rangle$). The diffusion operator then reflects all amplitudes about the new mean (dashed line), amplifying the target state's probability to 100% in a single iteration [6,5].

Repeating these operators drives the probability of measuring the target state close to 1. The precise number of optimal iterations k corresponds perfectly to $\mathcal{O}(\sqrt{N})$ [5,6]. Iterating past this threshold negatively rotates the state vector away from the solution, decreasing success probabilities in an effect termed "overcooking".

In essence, Grover's algorithm blindly acts upon amplitudes like a magnifying glass moving across a dark room. It does not understand what it is looking at, but iteratively brightens exactly the spot verified by the oracle. Because this mechanical process offers no structural shortcuts, a quadratic speedup is the absolute theoretical maximum. It is worth noting that this quadratic speedup is provably optimal. So no quantum algorithm can solve unstructured search faster than $\mathcal{O}(\sqrt{N})$. This bound arises because, in the absence of structure, quantum interference can only amplify the target amplitude gradually, with each iteration rotating the quantum state by an angle of approximately $1/\sqrt{N}$ in the relevant subspace. Crucially, while Grover's algorithm outperforms any classical approach, neither approach solves unstructured search in polynomial time with respect to the bit-length n of the input. Classical search requires $\mathcal{O}(2^n)$ steps, while Grover reduces this to $\mathcal{O}(2^{n/2})$. So Grover provides a provable quadratic speedup within an exponentially hard problem. [5,6]

6.2 Factorization in NP and co-NP: Shor's Algorithm

The integer factorization problem asks: given a composite integer N , find its prime factors. Classically, the best known algorithm is the General Number Field Sieve, which runs in sub-exponential time $\exp\left(\Theta\left(n^{1/3} \log^{2/3} n\right)\right)$ where n denotes the number of digits [6, p.218].

No classical polynomial algorithm is known.

Shor's algorithm solves factorization in polynomial time $\mathcal{O}(n^3)$ on a quantum computer [8]. The algorithm is not purely quantum but rather a hybrid classical-quantum workflow divided into distinct phases, as illustrated in Figure 3.[8]

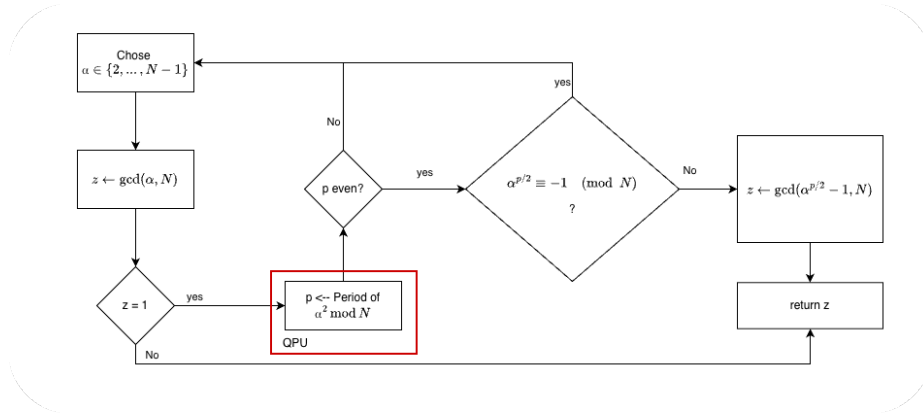


Fig. 3. The hybrid workflow of Shor's algorithm. The Quantum Processing Unit (QPU) is utilized exclusively to solve the period-finding bottleneck; all other verification and arithmetic reductions are performed classically. Adapted from [8].

As illustrated in Figure 3, the classical pre-processing and post-processing steps act purely as a wrapper. Once the classical computer ensures the number is neither even nor a prime power, it selects a random base ($a = 7$ in 4) and hands off the core problem. The quantum processing unit (QPU) is thereby delegated a single, highly specialized task: determining the periodicity of a complex function.

The QPU is given the classically defined function $f(x) = a^x \bmod N$ and a suitably large input range $Q \geq N^2 = (2^n)^2$ which is needed to ensure the period can be detected. It is expected to return a measurable resonance value from which the period p can be classically computed. The quantum execution achieves this across three core mechanical steps, which are visualized sequentially across the probability density charts in Figure 4:

Setup: Two quantum registers are allocated. Register $R1$ is initialized to a $|0\rangle^{\otimes 2^{2n}}$, while register $R2$ is initialized to the state $|0\rangle^{\otimes N_r}$ while $N_r \geq N$.

1) Superposition: Register $R1$ is placed in a uniform superposition of all Q possible inputs by applying the Hadamard gate to every single of its qubits to establish a superposition where each amplitude has the value $\frac{1}{\sqrt{Q}}$.

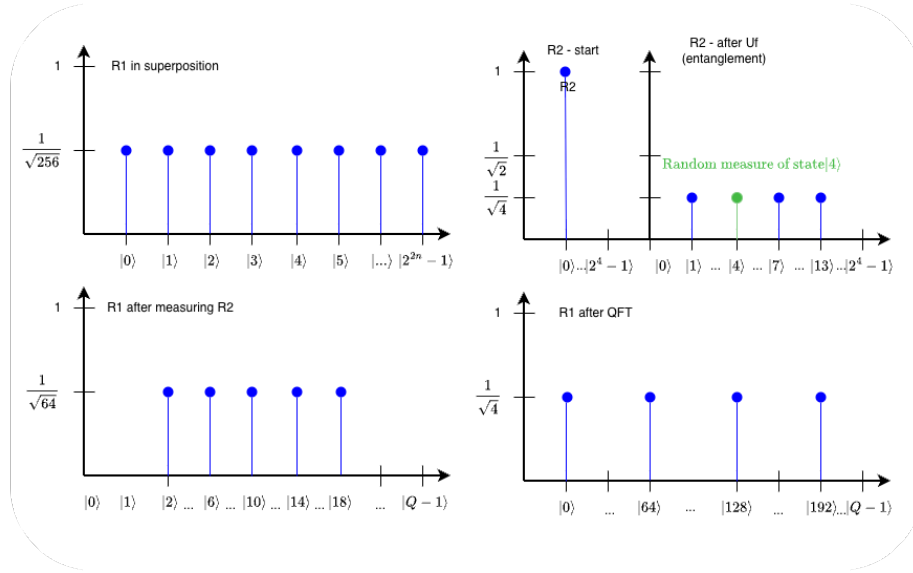


Fig. 4. Visualization of the amplitude development in R1 and R2 within the period-finding process for Shor's algorithm with $\alpha = 7$ & $N = 15$. Based on [6].

2) Quantum Oracle: A quantum oracle evaluates $f(x)$ for all inputs simultaneously and stores the results in $R2$, establishing massive entanglement to $R1$. [6, chapter 5.2]

3) Measurement-Induced Periodicity: Measuring $R2$ causes a partial collapse. Through entanglement, Register $R1$ instantaneously collapses into a state containing only the inputs x that yielded the measured outcome, encoding a periodic superposition where the distance between states is exactly p as you can see in the left bottom graphic in Figure 4. [6, chapter 5.2]

4) Quantum Fourier Transform (QFT): The QFT is applied to Register $R1$ to transform the periodic amplitude pattern into the frequency domain. As seen in the rightmost chart of Figure 4, this forces the amplitudes to undergo constructive interference at integer multiples of Q/p and destructive interference elsewhere. A subsequent measurement of $R1$ consequently yields a massive spike at a specific value k , from which the true mathematical period p can finally be recovered classically via the continued fraction algorithm applied to k/Q . [6, chapter 5.2]

In short, Shor's algorithm achieves its polynomial runtime, shifting the problem into BQP, by offloading only the computationally intractable periodic bottleneck to a quantum state. This is an explicit reliance on algebraic structure, vastly contrasting with Grover's unstructured approach.

6.3 The Boundary of Quantum Advantage: General NP-Complete Problems

The contrast between Grover’s quadratic speedup and Shor’s exponential breakthrough illustrates that quantum advantage requires more than classical difficulty. This structural dependency becomes especially apparent when considering general NP-complete optimization problems. NP-complete problems are the hardest problems in NP, and they are widely believed to lack the kind of periodic or algebraic regularity that the QFT exploits [2].

Attempting to solve an NP-complete optimization problem leaves two primary avenues. The first is applying unstructured quantum search directly to the complete solution space. While this provides a provable quadratic speedup over brute-force search, reducing a search space of $\mathcal{O}(2^n)$ to $\mathcal{O}(2^{n/2})$, the resulting complexity nevertheless remains exponentially intractable [3]. The second avenue explores heuristic quantum approaches, but they are outside the theoretical scope of this paper and therefore not analyzed here.

7 Discussion and Conclusion

The case studies reveal that quantum advantage is not determined by the classical hardness of a problem alone, but by the presence of exploitable mathematical structure.

This observation mathematically emerges when contrasting the complexity classes of these algorithms. Crucially, factorization sits in $\text{NP} \cap \text{co-NP}$. For factorization, both a "yes" instance (verifying factors) and a "no" instance (verifying primality) can be verified efficiently. If factorization were NP-complete, it would imply $\text{NP} = \text{co-NP}$, a scenario complexity theorists consider highly unlikely [2]. This indicates that factorization possesses a hidden mathematical structure that general NP-complete problems inherently lack. Shor’s algorithm translates this exact hidden algebraically periodic structure (modular exponentiation) into an exponential speedup via the QFT.

Grover’s quadratic speedup, by contrast, acts on unstructured search. It is the mathematically proven upper bound precisely because unstructured problems offer no regularity to exploit [3]. NP-complete tasks confirm this lack of speedup from the opposite direction: the absence of structure analogous to periodicity leaves quantum approaches without a provable asymptotic gain.

This leads to a broader principle: identifying and encoding exploitable structure is the central challenge in quantum algorithm design. Problems that lack such structure will likely remain resistant to exponential quantum acceleration regardless of future hardware progress. This intricately aligns with the widely held complexity-theoretic conjecture that $\text{NP} \not\subseteq \text{BQP}$.

Whether BQP contains any NP-complete problems remains one of the most consequential open questions in computer science. The evidence surveyed suggests that structure is the fundamental defining barrier to quantum utility.

References

1. Aaronson, S.: How much structure is needed for huge quantum speedups? (2022), <https://arxiv.org/abs/2209.06930>
2. Arora, S., Barak, B.: Computational Complexity: A Modern Approach. Cambridge University Press (2009)
3. Bennett, C.H., Bernstein, E., Brassard, G., Vazirani, U.: Strengths and weaknesses of quantum computing. *SIAM journal on Computing* **26**(5), 1510–1523 (1997)
4. Bernstein, E., Vazirani, U.: Quantum complexity theory. *SIAM Journal on Computing* **26**(5), 1411–1473 (1997)
5. Grover, L.K.: A fast quantum mechanical algorithm for database search. In: Proceedings of the 28th Annual ACM Symposium on Theory of Computing. pp. 212–219 (1996)
6. Nielsen, M.A., Chuang, I.L.: Quantum Computation and Quantum Information. Cambridge University Press, Cambridge, England (Dec 2010)
7. Rojas, R.: Konrad zuse’s legacy: The architecture of the z1 and z3. *IEEE Annals of the History of Computing* **19**(2), 5–16 (1997)
8. Shor, P.W.: Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Journal on Computing* **26**(5), 1484–1509 (Oct 1997). <https://doi.org/10.1137/s0097539795293172>, <http://dx.doi.org/10.1137/S0097539795293172>
9. Theis, T.N., Wong, H.S.P.: The end of moore’s law: A new beginning for information technology. *Computing in Science & Engineering* **19**(2), 41–50 (2017)
10. Turing, A.M.: On computable numbers, with an application to the entscheidungsproblem. *Proceedings of the London Mathematical Society* **42**, 230–265 (1936)